

COMPLETE DEVELOPER HANDBOOK

SQL & MySQL

Full Notes — Beginner to Pro, Explained Simply

Beginner Basics

Intermediate Queries

Pro-Level Database Design

Table of Contents

What is SQL & MySQL?	Beginner
Databases, Tables & Data Types	Beginner
Categories of SQL Commands	Beginner
Creating Databases & Tables (DDL)	Beginner
Constraints	Beginner
Inserting, Updating, Deleting Data (DML)	Beginner
SELECT & Filtering Data	Beginner
Sorting & Limiting Results	Beginner
Aggregate Functions	Intermediate
GROUP BY & HAVING	Intermediate
JOINS — The Complete Guide	Intermediate
Subqueries	Intermediate
Set Operations (UNION, INTERSECT)	Intermediate
String, Date & Math Functions	Intermediate
Primary Key vs Foreign Key Deep Dive	Intermediate
Normalization	Advanced
Indexes & Query Performance	Advanced
Views	Advanced
Stored Procedures & Functions	Advanced
Triggers	Advanced
Transactions & ACID Properties	Advanced
MySQL Storage Engines	Advanced
User Management & Security (DCL)	Advanced
Window Functions	Pro
Query Optimization Tips	Pro
Common Interview Questions	Pro
Quick Reference Cheat Sheet	Pro

1. What is SQL & MySQL?

SQL (Structured Query Language) is the standard language used to create, read, update, and delete data stored in a **relational database** (data organized into tables of rows and columns).

MySQL is one specific, hugely popular **Relational Database Management System (RDBMS)** — software that actually stores your data and lets you run SQL commands against it. SQL is the language; MySQL is one of many engines that understands it (others: PostgreSQL, Oracle, SQL Server, SQLite).

Think of SQL like English — a language for asking questions. MySQL is like a specific person (database engine) who understands that language and fetches/stores the data you ask for.

Relational Database Basics

- **Database:** A container holding related tables (e.g. "ecommerce_db")
- **Table:** A grid of data organized into rows and columns (e.g. "customers")
- **Row (Record):** One single entry in a table (e.g. one customer)
- **Column (Field):** One attribute of the data (e.g. "email", "name")

id	name	email
1	Riya Sharma	riya@example.com
2	Aman Verma	aman@example.com

2. Databases, Tables & Data Types

Common MySQL Data Types

Category	Type	Use For
Numeric	INT	Whole numbers (e.g. age, quantity)
	BIGINT	Very large whole numbers
	DECIMAL(p,s)	Exact decimals — money! (e.g. DECIMAL(10,2))
	FLOAT / DOUBLE	Approximate decimals (avoid for money)
String	VARCHAR(n)	Variable-length text up to n characters
	CHAR(n)	Fixed-length text (padded)
	TEXT	Long text (articles, descriptions)
	ENUM	One value from a predefined list
Date/Time	DATE	YYYY-MM-DD
	DATETIME	YYYY-MM-DD HH:MM:SS
	TIMESTAMP	Like DATETIME, auto-updates, timezone-aware
Boolean	BOOLEAN / TINYINT(1)	true/false flags

Tip: Always use **DECIMAL** for money, never **FLOAT** — floating point types can introduce tiny rounding errors that matter a lot in financial data.

3. Categories of SQL Commands

Category	Full Form	Purpose	Example Commands
DDL	Data Definition Language	Define/modify structure	CREATE, ALTER, DROP, TRUNCATE
DML	Data Manipulation Language	Modify data	INSERT, UPDATE, DELETE
DQL	Data Query Language	Read/query data	SELECT
DCL	Data Control Language	Permissions	GRANT, REVOKE
TCL	Transaction Control Language	Manage transactions	COMMIT, ROLLBACK, SAVEPOINT

4. Creating Databases & Tables (DDL)

Creating a Database

```
CREATE DATABASE ecommerce_db;
USE ecommerce_db;           -- switch to this database
SHOW DATABASES;            -- list all databases
DROP DATABASE ecommerce_db; -- deletes the database permanently
```

Creating a Table

```
CREATE TABLE customers (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(150) UNIQUE NOT NULL,
  age INT,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Altering a Table

```
ALTER TABLE customers ADD COLUMN phone VARCHAR(15);
ALTER TABLE customers MODIFY COLUMN age SMALLINT;
ALTER TABLE customers DROP COLUMN phone;
ALTER TABLE customers RENAME TO clients;
```

Deleting Tables/Data

```
DROP TABLE customers; -- deletes table AND structure
TRUNCATE TABLE customers; -- deletes ALL rows, keeps structure, resets AUTO_INCREMENT
```

DELETE vs TRUNCATE vs DROP: DELETE removes rows (can use WHERE, can rollback). TRUNCATE removes all rows instantly (can't use WHERE, faster, resets identity). DROP removes the entire table structure permanently.

5. Constraints

Constraints are rules enforced on table columns to maintain data accuracy and integrity.

Constraint	Purpose
PRIMARY KEY	Uniquely identifies each row; combines NOT NULL + UNIQUE
FOREIGN KEY	Links a column to another table's primary key
NOT NULL	Column cannot have an empty/NULL value
UNIQUE	All values in the column must be different
DEFAULT	Sets an automatic default value if none given
CHECK	Validates a condition (e.g. age >= 18)
AUTO_INCREMENT	Auto-generates the next sequential number (MySQL-specific)

```
CREATE TABLE orders (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  customer_id INT NOT NULL,  
  amount DECIMAL(10,2) CHECK (amount > 0),  
  status VARCHAR(20) DEFAULT 'pending',  
  FOREIGN KEY (customer_id) REFERENCES customers(id)  
);
```

6. Inserting, Updating, Deleting Data (DML)

INSERT

```
INSERT INTO customers (name, email, age)
VALUES ('Riya Sharma', 'riya@example.com', 28);

-- Insert multiple rows at once
INSERT INTO customers (name, email, age) VALUES
('Aman Verma', 'aman@example.com', 32),
('Priya Nair', 'priya@example.com', 25);
```

UPDATE

```
UPDATE customers
SET age = 29
WHERE id = 1;
```

Critical: Always use a WHERE clause with UPDATE and DELETE. Forgetting it will update/delete EVERY row in the table!

DELETE

```
DELETE FROM customers WHERE id = 3;
```


7. SELECT & Filtering Data

Basic SELECT

```
SELECT * FROM customers;           -- all columns
SELECT name, email FROM customers; -- specific columns
SELECT DISTINCT age FROM customers; -- unique values only
```

WHERE Clause — Filtering

```
SELECT * FROM customers WHERE age > 25;
SELECT * FROM customers WHERE age BETWEEN 20 AND 30;
SELECT * FROM customers WHERE name LIKE 'R%';      -- starts with R
SELECT * FROM customers WHERE name LIKE '%a';      -- ends with a
SELECT * FROM customers WHERE email IS NOT NULL;
SELECT * FROM customers WHERE age IN (25, 28, 32);
SELECT * FROM customers WHERE age NOT IN (25, 28);
SELECT * FROM customers WHERE age > 25 AND name LIKE 'R%';
```

Comparison & Logical Operators

Operator	Meaning
=, !=, <>	Equal, not equal
>, <, >=, <=	Greater/less than (or equal)
AND, OR, NOT	Combine conditions
LIKE / NOT LIKE	Pattern matching (% = any chars, _ = one char)
IN / NOT IN	Match against a list of values
BETWEEN	Value within a range (inclusive)
IS NULL / IS NOT NULL	Check for NULL values

8. Sorting & Limiting Results

```
SELECT * FROM customers ORDER BY age DESC;           -- descending
SELECT * FROM customers ORDER BY age ASC, name ASC; -- multi-column sort

SELECT * FROM customers LIMIT 5;                     -- first 5 rows
SELECT * FROM customers LIMIT 5 OFFSET 10; -- skip 10, then take 5 (pagination)
```

Pagination pattern: Page 3 with page size 10 → **LIMIT 10 OFFSET 20** (skip pages 1 & 2, i.e. 20 rows).

9. Aggregate Functions

Aggregate functions perform a calculation across multiple rows and return a single value.

Function	Purpose	Example
COUNT()	Number of rows	<code>SELECT COUNT(*) FROM customers;</code>
SUM()	Total of a numeric column	<code>SELECT SUM(amount) FROM orders;</code>
AVG()	Average value	<code>SELECT AVG(age) FROM customers;</code>
MAX()	Highest value	<code>SELECT MAX(amount) FROM orders;</code>
MIN()	Lowest value	<code>SELECT MIN(amount) FROM orders;</code>

10. GROUP BY & HAVING

GROUP BY groups rows sharing a value so aggregate functions can be applied per group. **HAVING** filters those groups (like WHERE, but for aggregated results).

```
-- Total orders and revenue per customer
SELECT customer_id, COUNT(*) AS total_orders, SUM(amount) AS total_spent
FROM orders
GROUP BY customer_id;

-- Only customers who spent more than 5000
SELECT customer_id, SUM(amount) AS total_spent
FROM orders
GROUP BY customer_id
HAVING total_spent > 5000
ORDER BY total_spent DESC;
```

WHERE vs HAVING: WHERE filters rows BEFORE grouping; HAVING filters groups AFTER aggregation. You cannot use an aggregate function (like SUM) inside WHERE.

Correct Clause Order

```
SELECT ... FROM ... WHERE ... GROUP BY ... HAVING ... ORDER BY ... LIMIT ...
```

11. JOINS — The Complete Guide

JOINS combine rows from two or more tables based on a related column (usually a foreign key).

Example setup: `customers(id, name)` and `orders(id, customer_id, amount)` — `orders.customer_id` references `customers.id`.

INNER JOIN — only matching rows

```
SELECT c.name, o.amount
FROM customers c
INNER JOIN orders o ON c.id = o.customer_id;
```

LEFT JOIN — all from left, matched or NULL from right

```
SELECT c.name, o.amount
FROM customers c
LEFT JOIN orders o ON c.id = o.customer_id;
-- Includes customers with ZERO orders (o.amount will be NULL)
```

RIGHT JOIN — all from right, matched or NULL from left

```
SELECT c.name, o.amount
FROM customers c
RIGHT JOIN orders o ON c.id = o.customer_id;
```

FULL OUTER JOIN — everything (MySQL: simulate with UNION)

```
-- MySQL doesn't support FULL OUTER JOIN directly:
SELECT c.name, o.amount FROM customers c LEFT JOIN orders o ON c.id = o.customer_id
UNION
SELECT c.name, o.amount FROM customers c RIGHT JOIN orders o ON c.id = o.customer_id;
```

SELF JOIN — table joined with itself

```
-- Find employees and their managers (same table)
SELECT e.name AS employee, m.name AS manager
FROM employees e
LEFT JOIN employees m ON e.manager_id = m.id;
```

Visual Summary

Join Type	Returns
INNER JOIN	Only rows with matches in BOTH tables
LEFT JOIN	ALL rows from left + matches from right (NULL if none)

RIGHT JOIN	ALL rows from right + matches from left (NULL if none)
FULL OUTER JOIN	ALL rows from both, matched where possible
CROSS JOIN	Every row from table A combined with every row from table B

12. Subqueries

A **subquery** is a query nested inside another query.

Subquery in WHERE

```
-- Customers who have placed at least one order
SELECT * FROM customers
WHERE id IN (SELECT DISTINCT customer_id FROM orders);
```

Subquery in SELECT

```
SELECT name,
       (SELECT COUNT(*) FROM orders WHERE orders.customer_id = customers.id) AS order_count
FROM customers;
```

Correlated Subquery

A subquery that references the outer query — runs once per outer row (can be slower).

```
-- Customers whose order total is above their own average order
SELECT * FROM orders o1
WHERE amount > (
  SELECT AVG(amount) FROM orders o2 WHERE o2.customer_id = o1.customer_id
);
```

Subquery vs JOIN

JOINS are usually faster and more readable for combining data. Subqueries are better for existence checks (`EXISTS` , `IN`) or when you need an aggregated value per row.

13. Set Operations

```
-- UNION: combines results, removes duplicates
SELECT name FROM customers
UNION
SELECT name FROM suppliers;

-- UNION ALL: combines results, KEEPS duplicates (faster)
SELECT name FROM customers
UNION ALL
SELECT name FROM suppliers;

-- INTERSECT (MySQL 8.0.31+): only rows in BOTH results
SELECT customer_id FROM orders_2024
INTERSECT
SELECT customer_id FROM orders_2025;
```

Rule: columns in each SELECT must match in number and compatible data type.

14. String, Date & Math Functions

String Functions

```
SELECT CONCAT(first_name, ' ', last_name) AS full_name FROM customers;
SELECT UPPER(name), LOWER(name) FROM customers;
SELECT LENGTH(name) FROM customers;
SELECT TRIM(' hello ');           -- 'hello'
SELECT SUBSTRING(name, 1, 3) FROM customers; -- first 3 characters
SELECT REPLACE(email, '@old.com', '@new.com') FROM customers;
```

Date Functions

```
SELECT NOW();           -- current date & time
SELECT CURDATE();       -- current date only
SELECT DATE_ADD(NOW(), INTERVAL 7 DAY);
SELECT DATEDIFF('2026-12-31', '2026-01-01'); -- days between
SELECT DATE_FORMAT(created_at, '%Y-%m-%d') FROM orders;
```

Math Functions

```
SELECT ROUND(19.987, 2); -- 19.99
SELECT CEIL(4.1);        -- 5
SELECT FLOOR(4.9);       -- 4
SELECT ABS(-15);         -- 15
```

15. Primary Key vs Foreign Key Deep Dive

Primary Key	Foreign Key
Uniquely identifies each row in ITS OWN table	References a primary key in ANOTHER table
Cannot be NULL	Can be NULL (if the relationship is optional)
Only one per table	A table can have multiple foreign keys
Enforces entity integrity	Enforces referential integrity

```
CREATE TABLE orders (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  customer_id INT,  
  FOREIGN KEY (customer_id) REFERENCES customers(id)  
    ON DELETE CASCADE  
    ON UPDATE CASCADE  
);
```

ON DELETE CASCADE: if the parent customer is deleted, their orders are automatically deleted too. Other options: **SET NULL**, **RESTRICT** (block deletion if child rows exist), **NO ACTION**.

16. Normalization

Normalization is the process of organizing tables to reduce data redundancy and improve integrity, by splitting data into related tables.

1NF — First Normal Form

Each column holds atomic (indivisible) values; no repeating groups/lists in a single cell.

```
-- Bad (1NF violation): phone = "9876543210, 9123456789"
-- Good: separate rows or a separate phones table
```

2NF — Second Normal Form

Must be in 1NF, and every non-key column must depend on the **WHOLE** primary key (relevant when using composite keys) — no partial dependency.

3NF — Third Normal Form

Must be in 2NF, and no non-key column should depend on another non-key column (no transitive dependency).

```
-- Bad: orders(id, customer_id, customer_city) — customer_city depends on customer_id, not on order id
-- Good: move customer_city into the customers table
```

Why Normalize?

- Eliminates duplicate data
- Prevents update/insert/delete anomalies
- Keeps data consistent

Trade-off: Highly normalized schemas need more JOINS to reconstruct data, which can hurt read performance. Sometimes **denormalization** (intentionally duplicating some data) is used for performance in read-heavy systems.

17. Indexes & Query Performance

An **index** is a special lookup structure that speeds up data retrieval — similar to an index at the back of a book.

```
CREATE INDEX idx_customer_email ON customers(email);
CREATE UNIQUE INDEX idx_unique_email ON customers(email);
DROP INDEX idx_customer_email ON customers;

-- See how MySQL will execute a query (uses index or not?)
EXPLAIN SELECT * FROM customers WHERE email = 'riya@example.com';
```

When to Use Indexes

- Columns frequently used in `WHERE`, `JOIN`, `ORDER BY`
- Foreign key columns (often indexed automatically)
- Columns with high "cardinality" (many distinct values) benefit most

Trade-off: Indexes speed up reads (SELECT) but slow down writes (INSERT/UPDATE/DELETE), since the index must also be updated. Don't over-index every column.

18. Views

A **view** is a saved, virtual table based on a SELECT query — it doesn't store data itself, just the query definition.

```
CREATE VIEW high_value_customers AS
SELECT c.id, c.name, SUM(o.amount) AS total_spent
FROM customers c
JOIN orders o ON c.id = o.customer_id
GROUP BY c.id
HAVING total_spent > 10000;

-- Use it just like a table:
SELECT * FROM high_value_customers WHERE total_spent > 20000;

DROP VIEW high_value_customers;
```

Why use views? Simplify complex/repeated queries, add a security layer (expose only certain columns), provide a consistent interface even if underlying tables change.

19. Stored Procedures & Functions

Stored Procedure

A saved block of SQL code that can be executed (called) repeatedly — can take parameters and doesn't have to return a value.

```
DELIMITER //
CREATE PROCEDURE GetCustomerOrders(IN cust_id INT)
BEGIN
    SELECT * FROM orders WHERE customer_id = cust_id;
END //
DELIMITER ;

CALL GetCustomerOrders(1);
```

Function

Similar to a procedure, but MUST return a single value and can be used inside a SELECT statement.

```
DELIMITER //
CREATE FUNCTION GetCustomerOrderCount(cust_id INT) RETURNS INT
DETERMINISTIC
BEGIN
    DECLARE order_count INT;
    SELECT COUNT(*) INTO order_count FROM orders WHERE customer_id = cust_id;
    RETURN order_count;
END //
DELIMITER ;

SELECT name, GetCustomerOrderCount(id) AS orders FROM customers;
```

20. Triggers

A **trigger** automatically executes a block of SQL in response to an INSERT, UPDATE, or DELETE event on a table.

```
DELIMITER //
CREATE TRIGGER before_order_insert
BEFORE INSERT ON orders
FOR EACH ROW
BEGIN
    IF NEW.amount <= 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Order amount must be positive';
    END IF;
END //
DELIMITER ;
```

Common Uses

- Auditing / logging changes (e.g. save old values to a history table)
- Enforcing complex business rules beyond simple CHECK constraints
- Auto-updating related/derived data (e.g. update a "last_modified" timestamp)

Overusing triggers can make debugging hard, since logic runs "invisibly" behind normal queries. Use sparingly and document well.

21. Transactions & ACID Properties

A **transaction** groups multiple SQL statements into a single all-or-nothing unit of work.

```
START TRANSACTION;

UPDATE accounts SET balance = balance - 500 WHERE id = 1;
UPDATE accounts SET balance = balance + 500 WHERE id = 2;

-- If both succeeded:
COMMIT;

-- If something went wrong:
ROLLBACK;
```

ACID Properties

Property	Meaning
Atomicity	All statements succeed together, or none do
Consistency	Database moves from one valid state to another, respecting all rules/constraints
Isolation	Concurrent transactions don't interfere with each other
Durability	Once committed, changes survive even a crash/power loss

Isolation Levels

Level	Prevents
READ UNCOMMITTED	Nothing — allows "dirty reads" of uncommitted data
READ COMMITTED	Dirty reads
REPEATABLE READ (MySQL default)	Dirty reads + non-repeatable reads
SERIALIZABLE	All — fully isolated but slowest

22. MySQL Storage Engines

Engine	Key Traits
InnoDB (default)	Supports transactions, foreign keys, row-level locking — use this for almost everything
MyISAM	Faster for read-heavy, no transaction support, table-level locking — largely legacy now
MEMORY	Data stored in RAM only — extremely fast but wiped on restart

```
CREATE TABLE orders (  
  id INT PRIMARY KEY  
) ENGINE=InnoDB;  
  
SHOW ENGINE INNODB STATUS;
```

23. User Management & Security (DCL)

```
-- Create a new user
CREATE USER 'app_user'@'localhost' IDENTIFIED BY 'StrongPassword123!';

-- Grant specific privileges
GRANT SELECT, INSERT, UPDATE ON ecommerce_db.* TO 'app_user'@'localhost';

-- Grant all privileges
GRANT ALL PRIVILEGES ON ecommerce_db.* TO 'admin_user'@'localhost';

-- Revoke privileges
REVOKE INSERT ON ecommerce_db.* FROM 'app_user'@'localhost';

-- Apply changes
FLUSH PRIVILEGES;

DROP USER 'app_user'@'localhost';
```

Security best practices: Never use the root user for application connections. Grant only the minimum privileges needed (Principle of Least Privilege). Always use parameterized queries in application code to prevent SQL injection — never build queries with raw string concatenation of user input.

SQL Injection Example (what NOT to do)

```
-- DANGEROUS: directly inserting user input into a query string
String query = "SELECT * FROM users WHERE username = '" + userInput + "'";
-- If userInput = "' OR '1'='1", this returns ALL users!

-- SAFE: use parameterized/prepared statements
SELECT * FROM users WHERE username = ?; -- bind userInput safely
```

24. Window Functions

Window functions perform calculations across a set of rows related to the current row, WITHOUT collapsing them into a single output row (unlike GROUP BY).

```
-- Rank customers by total spend
SELECT name, total_spent,
       RANK() OVER (ORDER BY total_spent DESC) AS spend_rank
FROM customer_totals;

-- Running total per customer, ordered by date
SELECT customer_id, order_date, amount,
       SUM(amount) OVER (PARTITION BY customer_id ORDER BY order_date) AS running_total
FROM orders;
```

Function	Purpose
ROW_NUMBER()	Unique sequential number per row
RANK() / DENSE_RANK()	Ranking, with/without gaps for ties
LAG() / LEAD()	Access previous/next row's value
SUM()/AVG() OVER()	Running totals/averages without collapsing rows

25. Query Optimization Tips

- Avoid `SELECT *` — only fetch columns you actually need
- Use `EXPLAIN` to check if a query is using indexes properly
- Index columns used in WHERE, JOIN, and ORDER BY frequently
- Avoid functions on indexed columns in WHERE (e.g. `WHERE YEAR(created_at) = 2026` can't use an index — use a range instead: `WHERE created_at >= '2026-01-01'`)
- Use `LIMIT` for large result sets, especially during development/testing
- Prefer JOINS over correlated subqueries where possible
- Batch large INSERT/UPDATE operations instead of row-by-row loops
- Normalize to reduce redundancy, but consider denormalizing hot read paths for performance

26. Common Interview Questions

Q1: Difference between WHERE and HAVING?

WHERE filters rows before grouping/aggregation; HAVING filters groups after aggregation, and can use aggregate functions.

Q2: Difference between DELETE, TRUNCATE, and DROP?

DELETE removes rows (can filter with WHERE, is logged, can rollback). TRUNCATE removes all rows instantly (resets auto-increment, minimal logging). DROP removes the entire table structure.

Q3: What is a primary key vs a unique key?

A table can have only ONE primary key (cannot be NULL), but MULTIPLE unique keys (can allow one NULL, depending on DB).

Q4: Explain the difference between INNER JOIN and LEFT JOIN.

INNER JOIN returns only matching rows from both tables. LEFT JOIN returns all rows from the left table, with NULLs where there's no match on the right.

Q5: What is normalization and why is it important?

The process of structuring tables to reduce redundancy and improve data integrity by splitting data logically across related tables.

Q6: What are ACID properties?

Atomicity, Consistency, Isolation, Durability — the four guarantees a database transaction system provides.

Q7: How would you prevent SQL injection?

Always use parameterized/prepared statements instead of concatenating raw user input into SQL strings; validate and sanitize input; apply least-privilege database permissions.

Q8: What's the difference between a clustered and non-clustered index?

A clustered index determines the physical order of data in the table (only one per table, e.g. the primary key in InnoDB). A non-clustered index is a separate structure pointing back to the row — a table can have many.

27. Quick Reference Cheat Sheet

Task	Command
Create database	<code>CREATE DATABASE name;</code>
Create table	<code>CREATE TABLE t (col TYPE, ...);</code>
Insert data	<code>INSERT INTO t (cols) VALUES (...);</code>
Read data	<code>SELECT cols FROM t WHERE ...;</code>
Update data	<code>UPDATE t SET col=val WHERE ...;</code>
Delete data	<code>DELETE FROM t WHERE ...;</code>
Join tables	<code>SELECT ... FROM a JOIN b ON a.id=b.a_id;</code>
Group + aggregate	<code>SELECT col, COUNT(*) FROM t GROUP BY col HAVING ...;</code>
Sort & limit	<code>ORDER BY col DESC LIMIT n OFFSET m;</code>
Add index	<code>CREATE INDEX idx ON t(col);</code>
Start transaction	<code>START TRANSACTION; ... COMMIT/ROLLBACK;</code>
Grant access	<code>GRANT privileges ON db.* TO 'user'@'host';</code>

You've now covered SQL & MySQL from the ground up to professional database design, performance, and security. Practice by designing and querying a small real project (e.g. a "Library" or "E-commerce" database) using joins, constraints, indexes, and at least one stored procedure/trigger — that's the fastest way to make it stick.